[beacon]

Your academic data, one dashboard away.

AI-powered insights from Brightspace, unified into a beautiful dashboard built for Purdue students.

CNIT 566 · Next.js 16 · Supabase · Anthropic · TypeScript · Apr 2026

AI-native · RLS-isolated · Streaming · Mobile-ready

the problem

College dashboards weren't built for how students actually work.

Brightspace is a system of record, not a workspace. Every piece of information lives behind another click.

- 4+ clicks to see a grade
- 6 courses × 5 tabs = 30 surfaces
- Zero proactive nudges

Grades hidden three clicks deep
Open course → grades tab → scroll

Assignments scattered per course
No cross-course upcoming view

Deadlines tracked manually
Students copy dates into Notion / paper

Announcements siloed
Must visit every course weekly

No weekly workload overview
Can't plan the week at a glance

Contact info lives in PDFs
Office hours buried in syllabi

the solution

One surface.
Every course. Every deadline. Every answer.

Beacon syncs Brightspace into a local-first dashboard and layers AI on top — so the next action is always obvious.

01

Sync

Pull courses, assignments, grades, content, announcements from Brightspace Valence API.

OAuth + DUO · Bookmark paging · Background job

02

Surface

Unified dashboard, calendar, per-course tabs, grade analytics — all powered by local Postgres.

<100ms reads · Offline-capable · Mobile-first

03

Ask

Natural-language queries over structured academic context with streaming token responses.

SSE streaming · Dual-scoped · Rate-limited

what it does

Eight features.
One coherent surface.

AI daily briefing

Streaming summaries in 3 sections: due soon, this week, heads-up. Cached per day, <80 words.

`/api/briefing · SSE`

Ask Beacon

Chat over full academic context. Global + per-course scopes. History persisted per user.

`/api/ask · 20 req/min`

Unified calendar

Month + week views. Assignment dots color-coded by course. Click to jump to course detail.

`/calendar`

Grade analytics

Weighted calculations. Trend detection. AI-generated insight under 80 words on weak spots.

`/api/course/grade-insight`

Content browser

Hierarchical modules + topics. Direct links out to Brightspace resources.

`content_modules / topics`

Instructor card

AI parses announcements to extract name, email, office, and hours into structured JSON.

`/api/course/extract-info`

Daily checklist

Auto-populated from upcoming assignments. Custom tasks allowed. Resets every day.

`localStorage scoped by date`

Announcements feed

Cross-course announcement stream. HTML cleaned, ordered by recency.

`announcements table`

tech stack

Modern, typed, no framework sprawl.

FRONTEND

- Next.js 16.1.7 App Router
- React 19.2.3
- TypeScript 5 (strict)
- Tailwind CSS v4
- Framer Motion 12
- Lucide icons 0.577
- Zustand 5 state
- react-markdown + GFM

BACKEND

- Next.js API routes (21)
- @supabase/ssr 0.9
- @supabase/supabase-js 2.99
- Row-Level Security
- SSE async generators
- In-memory rate limiter
- Middleware auth
- Service-role admin client

DATA

- Brightspace Valence API
- LP version 1.40
- LE version 1.74
- Bookmark pagination
- Enrollment type 3 filter
- Playwright 1.58 (DUO)
- Exponential backoff on 429
- 24-hour cookie expiry

AI

- Anthropic SDK 0.90
- api.anthropic.com endpoint
- Claude Haiku 4.5 model
- Streaming + non-stream
- 6 system prompt templates
- Structured academic context
- Conversation history (10-msg)
- 10/min & 20/min buckets

architecture

One codebase.
End to end.

Browser
React 19 · Tailwind · Framer Motion
Server components hydrate on load. Client components for interactive UI.
01

Next.js App Router
21 API routes · middleware auth · SSR
Single deploy. Edge-ready. Streaming SSE endpoints for AI.
02

AI layer
Anthropic streaming + context builder
buildAcademicContext → system prompt → async generator → SSE stream.
03

Supabase Postgres
8 tables · RLS on every row
auth.uid() filter on every query. Service role only for internal jobs.
04

Brightspace Valence
OAuth + Playwright fallback
LP 1.40 / LE 1.74. Cookie session. Bookmark pagination loops.
05

database schema

Eight tables.
RLS on every row.

Every row is scoped to `auth.uid()`. The database enforces isolation — the API layer can't leak data even if it tries.

RLS enforced | # 6 indexes | 2 migrations

users	Auth + Brightspace tokens + sync progress	1 / user
courses	Enrollments with current_grade numeric	~6–8 / user
assignments	Dropbox + quizzes with due_date, points, weight	~50–200
content_modules	Hierarchical modules, self-referential FK	~30 / course
content_topics	Individual items inside modules	~100 / course
announcements	Course news, HTML-cleaned, ordered by date	~20 / course
briefings	Cached AI briefings, unique per user/day	1 / user / day
chat_messages	Scoped chat (null course_id = global)	unbounded

ai layer

Not a chatbot wrapper.
A context engine.

Streaming SSE

Async generators wrapped in Server-Sent Events. Tokens stream token-by-token to the browser — no blank wait, no buffering.

`streamToSSEResponse()`

Structured context

buildAcademicContext() formats courses, upcoming work, recent grades, and announcements into markdown sections before prompting.

`src/lib/ai/prompts.ts`

Dual-scoped chat

chat_messages.course_id IS NULL for global; set for per-course. Same table, same RLS — different scope via partial index.

`idx_chat_messages_global`

Six specialized prompts

Briefing · NLQ · course-NLQ · assignment analysis · grade insight · summarization — each tuned under a strict word budget.

`171 lines of prompt eng.`

Tight word budgets

Briefings < 80 words. NLQ < 100 unless a table is needed. Grade insight < 80. Forces the model to be useful, not verbose.

`system prompt constraints`

Rate limits

Per-user, per-endpoint buckets in memory. Briefing 10/min, Ask 20/min. Auto-cleanup every 60s. Zero DB calls.

`lib/rateLimit.ts`

streaming pattern

Async generator
→ Server-Sent Events.

src/lib/ai/client.ts

```
export async function* generateCompletionStream(systemPrompt, userMessage) {
  const stream = await anthropic.messages.stream({
    model: 'claude-haiku-4-5',
    max_tokens: 2000,
    system: systemPrompt,
    messages: [{ role: 'user', content: userMessage }],
  });

  for await (const event of stream) {
    if (event.type === 'content_block_delta') yield event.delta.text; // one token per yield
  }
}

export function streamToSSEResponse(generator) {
  const encoder = new TextEncoder();
  const readable = new ReadableStream({
    async start(controller) {
      for await (const token of generator) {
        controller.enqueue(encoder.encode(`data: ${JSON.stringify({ content: token })}\n\n`));
      }
      controller.enqueue(encoder.encode('data: [DONE]\n\n'));
      controller.close();
    },
  });
  return new Response(readable, { headers: { 'Content-Type': 'text/event-stream' } });
}
```

One helper. Reused by `/api/briefing`, `/api/ask`, and `/api/course/grade-insight`.

brightspace integration

OAuth, pagination, and a headless browser fallback.

Purdue Brightspace requires DUO 2FA. When OAuth isn't approved, a Playwright flow captures session cookies server-side.

LP v1.40 | LE v1.74 | OAuth 2.0 | Bookmark paging | Playwright fallback

01	Auth	Playwright captures session cookies through DUO 2FA. 24-hour expiry window.
02	Enrollments	Filter to Type 3 (Course Offering). Bookmark-paginated via PagingInfo.HasMoreItems loop.
03	Per course	Dropbox folders + quizzes + grade values + final grade + content tree + announcements.
04	Persist	Upsert into Supabase. Progress written to users.sync_progress jsonb. Polled from UI.
05	Resilience	Exponential backoff on 429. 302→login redirect detected as session expiry. 3 retries.

security & isolation

Defense in depth.
Postgres as the last line.

Row-Level Security

Every table has a policy: id = auth.uid(). Enforced by Postgres — API bugs can't leak cross-user data.

`USING (user_id = auth.uid())`

Token storage

Brightspace session cookies stored server-side, never exposed to the client. Service-role key only on the server.

`users.brightspace_session_cookies (jsonb)`

Rate limiting

Per-user, per-endpoint. 10/min briefing, 20/min ask. Prevents LLM cost blowouts and abuse.

`checkRateLimit(userId, endpoint)`

Auth middleware

Every API route validates Supabase JWT before touching data. Anonymous requests are rejected at the edge.

`middleware.ts`

No secret leakage

Environment variables for all credentials. .env.local gitignored. No keys in client bundles.

`.env.local + NEXT_PUBLIC_ prefix`

Session expiry

Brightspace cookies checked for 24-hour window. Expired sessions force re-auth with clear UI.

`getValidToken(userId)`

product surfaces

Six routes.
Every student task covered.

Dashboard `/dashboard`
Briefing, checklist, deadlines, recent updates.

Course detail `/course/[id]`
7 tabs: overview, assignments, grades, calendar, content, chat, announcements.

Ask Beacon `/ask`
Global streaming chat with suggestion pills + history.

Calendar `/calendar`
Month + week views with color-coded assignment dots.

Settings `/settings`
Theme, Brightspace auth, data export, notifications.

Landing `/`
Auth form with feature showcase and animated background.

design system

Monochrome-first.
Content is the color.

Palette

oklch achromatic. Zero chroma in the base. Grade colors for semantic accent only.

Typography

System fonts. font-light headlines. font-mono brand + labels. Never bold.

Aa

[beacon]

300 · 400 · 500

Components

Pills everywhere. rounded-2xl cards. Soft focus rings. No resting shadow.

Primary

Ghost

Motion

Framer Motion. Entrance y=20 once. Hover scale 1.02 on buttons, y=-8 on cards.

3 themes

by the numbers

What's inside.

89
TypeScript files

21
API route handlers

18
React components

8
database tables

6
AI prompt templates

15
Brightspace endpoints

3
streaming SSE routes

2
auth flows

user journey

From zero to briefing
in under two minutes.

Sign in <10s
Supabase auth — email, password, or Google SSO.
step 1

Connect Brightspace ~30s
Browser-based DUO 2FA. Cookies stored server-side.
step 2

Initial sync ~60s
Courses, assignments, grades, content, announcements.
step 3

Daily briefing <3s
AI streams a personalized summary on dashboard load.
step 4

Ask anything real-time
Scoped chat over real academic data.
step 5

what's interesting

Six decisions worth pointing out.

Streaming SSE from async generators

AI completions wrapped in a single streamToSSEResponse() helper — reused across briefing, ask, grade-insight.

Dual-scoped chat via nullable FK

course_id IS NULL for global chat, set for per-course. Partial indexes on both. One table, zero branching.

Structured context builder

User data → markdown sections → system prompt. The model sees clean, scoped context instead of raw JSON dumps.

AI as a data extractor

Professor card uses LLM to extract structured JSON (name, email, office, hours) from unstructured announcements.

Playwright as an auth fallback

When OAuth approval stalls, a headless browser captures the session through DUO so the product still ships.

Grade-aware semantic palette

Theme defines grade-a through grade-d as oklch colors. A-grade = emerald, D = red. Semantic, not decorative.

what's next

Ship, measure, then expand scope.

NEAR TERM

- Push notifications for due-soon work
- PDF syllabus upload + summary
- Weekly retrospective briefing
- iOS PWA home-screen polish
- Email digest opt-in

FALLBACK PLANS

- A — manual CSV / PDF upload
- B — Chrome-extension DOM scraper
- C — mock-data demo path

Every fallback keeps the same architecture — only the data source changes. The AI layer, DB schema, and UI all stay identical.

[beacon]

Thank you.
Questions?

Try it at `/dashboard` · ask it at `/ask`

Open dashboard | Ask Beacon | See calendar

AI-native · RLS-isolated · Streaming · Mobile-ready · Built for Purdue